PONTIFICIA UNIVERSIDAD CATÓLICA MADRE Y MAESTRA

Facultad de Ciencias e Ingeniería

Escuela de Ingeniería en Computación y Telecomunicaciones

INFORME TÉCNICO FINAL

Sistema de Gestión de Rutas de Transporte Público

Asignatura: Algoritmos Clásicos y Estructuras de Datos

Docente: Freddy Antonio Peña Peguero

Proyecto Final | Año 2026

Integrantes del Equipo

Anabelle Liberato Torres ID:10159061

Abel Enrhique García ID:10160649

1. Introducción

El presente informe documenta el diseño, implementación y evaluación del Sistema de Gestión de Rutas de Transporte Público, desarrollado como proyecto final de la asignatura Algoritmos Clásicos y Estructuras de Datos de la Pontificia Universidad Católica Madre y Maestra (PUCMM).

El sistema tiene como propósito central ofrecer a los usuarios una herramienta interactiva para gestionar y optimizar rutas dentro de una red de transporte urbano. Permite agregar, modificar y eliminar paradas y rutas, así como calcular los caminos óptimos entre dos puntos considerando criterios de tiempo, costo, distancia y número de transbordos. La solución fue construida íntegramente en Java, empleando JavaFX para la interfaz gráfica, la librería SmartGraph para la visualización del grafo, y PostgreSQL como mecanismo de persistencia.

La gestión de dependencias y la construcción del proyecto se realizaron mediante Apache Maven, una herramienta de automatización ampliamente utilizada en el ecosistema Java. Maven centraliza en un único archivo de configuración (pom.xml) todas las dependencias externas del proyecto, como el driver JDBC de PostgreSQL, la librería SmartGraph y los módulos de JavaFX, y se encarga de descargarlas, compilarlas y empaquetarlas de forma automática. Gracias a Maven, cualquier integrante del equipo puede clonar el repositorio y ejecutar el proyecto sin necesidad de configurar manualmente las librerías, lo que facilita la colaboración y garantiza que todos trabajen con las mismas versiones de cada dependencia.

El repositorio del proyecto refleja el trabajo colaborativo de ambos integrantes a través de múltiples commits a lo largo del desarrollo, cubriendo desde la definición inicial de la estructura de datos hasta las correcciones visuales y la integración de la base de datos.

2. Arquitectura y Estructura del Proyecto

El proyecto sigue una arquitectura en capas claramente definida, separando la lógica del dominio, la capa de datos y la capa visual. Esta organización favorece la modularidad, la mantenibilidad y la escalabilidad del sistema.

2.1 Organización de Paquetes

Paquete	Responsabilidad
com.rutas.logico.modelo	Clases del dominio: Parada, Ruta, GrafoTransporte, Criterio, TipoParada
com.rutas.logico.algoritmos	Implementación de Dijkstra, Bellman-Ford, Floyd-Warshall y Kruskal
com.rutas.logico.crud	Lógica CRUD sobre el grafo: ParadaCrud, RutaCrud
com.rutas.logico.database	Conexión JDBC, inicialización de BD, StatementService, RowMapper
com.rutas.logico.database.repositorios	ParadaRepositorio, RutaRepositorio (acceso a PostgreSQL)
com.rutas.logico.database.mapper	ParadaRowMapper, RutaRowMapper (mapeo ResultSet)
com.rutas.visual.controladores	Controladores JavaFX: Mapa, Paradas, Rutas, Registro, Principal, Menú
com.rutas.servicios	ServicioGrafo: singleton que provee el grafo compartido a toda el programa

2.2 Patrón de Diseño General

La aplicación adopta un patrón similar a MVC (Model-View-Controller): el modelo está representado por las clases del paquete logico.modelo, las vistas por los archivos FXML del paquete resources,view, y los controladores por las clases del paquete visual.controladores. La comunicación entre capas se realiza a través del singleton ServicioGrafo, que actúa como punto central de acceso al grafo en memoria, garantizando consistencia entre todas las vistas.

3. Estructura de Datos: Grafo Dirigido

El núcleo del sistema es un grafo dirigido (dígrafo) implementado mediante la clase GrafoTransporte. Esta clase envuelve la implementación DigraphEdgeList de la librería SmartGraph, que internamente utiliza listas de adyacencia —la estructura más eficiente para representar redes de rutas densas.

3.1 Clase GrafoTransporte

GrafoTransporte mantiene dos estructuras internas principales: el dígrafo de SmartGraph (Digraph<Parada, Ruta>) y un HashMap<String, Vertex<Parada>> que permite la búsqueda de vértices por código de parada en O(1).

Método	Descripción	Complejidad
agregarParada(Parada)	Inserta un vértice en el dígrafo; ignora duplicados	O(1)
eliminarParada(Parada)	Elimina el vértice y todas sus aristas incidentes	O(E)
existeParada(Parada)	Verifica existencia por código en el HashMap	O(1)
agregarRuta(Ruta)	Inserta una arista dirigida entre dos vértices existentes	O(1)
eliminarRuta(Ruta)	Busca la arista por ID y la elimina del dígrafo	O(E)
modificarRuta(Ruta, Ruta)	Elimina la arista original e inserta la nueva	O(E)
getRutas(Parada)	Obtiene las aristas salientes de un vértice (vecinos)	O(deg)
esConexo()	BFS doble (adelante + grafo inverso) para verificar fuerte conexidad	O(V+E)
copiar()	Crea una copia superficial del grafo para simulaciones	O(V+E)

3.2 Verificación de Conexidad

Un aspecto destacable es el método esConexo(), que implementa dos BFS secuenciales para verificar la conexidad fuerte del dígrafo. El primero recorre el grafo hacia adelante desde un vértice inicial; el segundo recorre el grafo inverso (usando un mapa de predecesores). Si algún vértice no es alcanzado en cualquiera de los dos recorridos, se retorna esa parada como problemática. Este mecanismo se usa antes de cada búsqueda de ruta y al modificar el grafo, garantizando que el sistema siempre opere sobre una red correctamente conectada.

3.3 Clases del Dominio

Clase	Atributos Clave	Rol en el sistema
Parada	codigo, nombreParada, tipo (TipoParada), ubicacion	Vértice del grafo — representa cada punto de la red
Ruta	id, nombre, origen (Parada), destino (Parada), pesos (Map<Criterio,Object>)	Arista dirigida — representa la conexión entre paradas
Criterio	TIEMPO, COSTO, DISTANCIA, TRANSBORDOS	Enum que define los criterios de optimización
TipoParada	BUS, METRO, TREN, etc.	Enum que clasifica el tipo de transporte de cada parada
GrafoTransporte	Digraph + HashMap de vértices	Contenedor central — expuesto como singleton vía ServicioGrafo

4. Algoritmos de Optimización Implementados

4.1 Dijkstra

El algoritmo de Dijkstra es el principal motor de búsqueda del sistema. Está implementado en la clase com.rutas.logico.algoritmos. Dijkstra con el método estático ejecutar(). Opera sobre el GrafoTransporte y acepta cualquier criterio del enum Criterio como peso de las aristas.

Funcionamiento implementado:

- Se inicializan todos los costos a Double.MAX_VALUE, excepto el origen (costo = 0.0).
- Se utiliza una PriorityQueue<Parada> que ordena los nodos por costo acumulado.
- En cada iteración se extrae el nodo de menor costo, se marca como visitado y se relajan sus vecinos.
- Al encontrar el destino, se reconstruye el camino recorriendo el mapa anterior[] hacia atrás.
- Complejidad: O((V + E) log V) con la cola de prioridad implementada.

4.2 Búsqueda de Rutas Alternativas

La clase Dijkstra también expone el método ejecutarAlternativo(), que recibe la lista de caminos previos ya encontrados. Este método vuelve a ejecutar Dijkstra excluyendo los caminos ya conocidos, permitiendo al usuario descubrir rutas secundarias entre el mismo par de paradas. El controlador MapaController acumula todos los caminos en la lista caminosPrevios y los pasa como parámetro en cada llamada sucesiva.

4.3 Bellman-Ford (Integrado en la Interfaz)

El sistema incluye la clase BellmanFord en el paquete com.rutas.logico.algoritmos. En la interfaz visual (MapaController) el usuario puede seleccionar entre Dijkstra y Bellman-Ford mediante el ComboBox cmbAlgoritmo. Bellman-Ford es especialmente relevante para el caso contemplado por el proyecto de pesos negativos en aristas, como descuentos en tarifas. Los Spinners de registro de rutas permiten valores negativos (mínimo: -9999) para habilitar este escenario.

4.4 Floyd-Warshall

El algoritmo de Floyd-Warshall está implementado en la clase com.rutas.logico.algoritmos.FloydWarshall con el método estático ejecutar(). A diferencia de Dijkstra y Bellman-Ford, que calculan el camino óptimo desde un origen hacia un destino

específico, Floyd-Warshall opera sobre todos los pares de paradas del grafo simultáneamente, construyendo una matriz de distancias mínimas completa.

Funcionamiento implementado:

- Se inicializan dos matrices NxN: dist[i][j] con el peso de la arista directa si existe, 0 si i == j, e infinito en caso contrario; y next[i][j] que registra el siguiente nodo en el camino óptimo de i hacia j.
- Se ejecuta el triple bucle sobre todos los nodos intermedios k, y para cada par (i, j) se evalúa si pasar por k reduce el costo: si dist[i][k] + dist[k][j] < dist[i][j], se actualiza la distancia y se registra next[i][j] = next[i][k].
- Una vez completadas las matrices, se reconstruye el camino entre el origen y el destino seleccionados siguiendo la tabla next[] desde el origen hasta llegar al destino. Si dist[origen][destino] permanece infinito, se retorna null.
- Complejidad: $O(V^3)$ en tiempo y $O(V^2)$ en espacio. Es el algoritmo más costoso del sistema, pero el único que garantiza el camino óptimo entre cualquier par de paradas sin importar la topología del grafo.

En la interfaz visual, el usuario selecciona Floyd-Warshall desde el ComboBox cmbAlgoritmo junto con el criterio de optimización deseado. El sistema requiere que se hayan seleccionado paradas de origen y destino mediante doble clic en el mapa. Al ejecutarse, el camino óptimo entre esas dos paradas se resalta visualmente sobre el grafo, al igual que con Dijkstra y Bellman-Ford. El botón de ruta alternativa permanece desactivado para este algoritmo.

4.5 Resumen Comparativo de Algoritmos

Algoritmo	Caso de uso	Complejidad	Implementado
Dijkstra	Ruta óptima con pesos positivos (tiempo, costo, distancia, transbordos)	O((V+E) log V)	Sí — clase Dijkstra
Bellman-Ford	Ruta óptima con posibles pesos negativos (descuentos)	O(V × E)	Sí — clase BellmanFord
Floyd-Warshall	Camino mínimo entre todos los pares de paradas	$O(V^3)$	Sí — clase FloydWarshall

5. Capa de Lógica de Negocio (CRUD)

Las clases ParadaCrud y RutaCrud encapsulan toda la lógica de negocio relacionada con la gestión de paradas y rutas. Actúan como intermediarias entre los controladores de la vista y el grafo, garantizando validaciones antes de modificar la estructura del dígrafo.

5.1 ParadaCrud

Método	Validaciones	Retorno
insertarParada(nombre, tipo, ubicacion)	nombre no nulo ni en blanco; tipo no nulo; autogenera código incremental	boolean
modificarParada(parada, nombre, tipo, ubicacion)	Delega en GrafoTransporte.modificarParada()	boolean
eliminarParada(parada)	Elimina del grafo junto con todas las rutas asociadas	boolean
listarParadas()	Retorna la lista de paradas del grafo	List<Parada>

5.2 RutaCrud

Método	Validaciones	Retorno
agregarRuta(nombre, origen, destino, ...)	Mínimo 2 paradas; origen ≠ destino; no debe existir ruta entre ese par	boolean
modificarRuta(ruta, nombre, origen, destino, ...)	Si cambian las paradas, simula en copia del grafo y verifica conexidad	void
eliminarRuta(ruta)	Elimina la arista del dígrafo	void
listarRutas()	Agrega las rutas de todas las paradas	List<Ruta>

Una validación especialmente importante en RutaCrud.modificarRuta() es la verificación de conexidad antes de aplicar cambios al par origen-destino: el sistema crea una copia del grafo, aplica la modificación propuesta y llama a esConexo(). Si el resultado es un grafo no conexo, la operación se rechaza y se notifica al usuario la parada problemática.

6. Capa de Persistencia (Base de Datos)

El sistema implementa persistencia completa mediante PostgreSQL, con una capa de acceso a datos propia basada en el patrón Repository y el patrón RowMapper. Esto desacopla la lógica SQL de las entidades del dominio.

6.1 Esquema de Base de Datos

La clase DatabaseInitializer crea automáticamente las tablas al iniciar la aplicación si no existen. El esquema es el siguiente:

```
CREATE TABLE IF NOT EXISTS paradas (
    codigo    VARCHAR(20)  NOT NULL,
    nombre    VARCHAR(100) NOT NULL,
    tipo      VARCHAR(50)  NOT NULL,
    ubicacion VARCHAR(200),
    CONSTRAINT paradas_pkey PRIMARY KEY (codigo)
);

CREATE TABLE IF NOT EXISTS rutas (
    id          VARCHAR(20)  NOT NULL,
    nombre      VARCHAR(100) NOT NULL,
    origen      VARCHAR(20)  NOT NULL REFERENCES paradas(codigo) ON DELETE
CASCADE,
    destino     VARCHAR(20)  NOT NULL REFERENCES paradas(codigo) ON DELETE
CASCADE,
    tiempo      DOUBLE PRECISION,
    costo       DOUBLE PRECISION,
    distancia   DOUBLE PRECISION,
    transbordos INTEGER,
    CONSTRAINT rutas_pkey  PRIMARY KEY (id),
    CONSTRAINT rutas_unica UNIQUE (origen, destino)
);
```

6.2 Componentes de Acceso a Datos

Clase	Rol
JdbcConnection	Proveedor de conexiones JDBC a PostgreSQL (singleton)
StatementService<T>	Servicio genérico que ejecuta PreparedStatements y delega el mapeo a RowMapper
PreparedStatementMapper<T>	Interfaz funcional que define la query SQL y el bind de parámetros

Clase	Rol
RowMapper<T>	Interfaz que convierte un ResultSet en una entidad del dominio
ParadaRowMapper	Mapea ResultSet → Parada; define la query SELECT de paradas
RutaRowMapper	Mapea ResultSet → Ruta con sus cuatro pesos; define la query SELECT de rutas
ParadaRepositorio	Operaciones CRUD SQL para paradas (insertar, actualizar, eliminar, listarTodas)
RutaRepositorio	Operaciones CRUD SQL para rutas (insertar, actualizar, eliminar, listarTodas)

La integración entre la capa de datos y la capa lógica es completa: RutaCrud (versión con persistencia) llama al repositorio correspondiente en cada operación, garantizando que los cambios en el grafo en memoria queden también registrados en la base de datos.

7. Interfaz Gráfica (JavaFX + SmartGraph)

La interfaz del sistema fue desarrollada íntegramente con JavaFX, utilizando archivos FXML para declarar la estructura de cada vista y clases controladoras para manejar la lógica de interacción. La visualización del grafo se apoya en la librería de código abierto SmartGraph (brunomnsilva), que provee renderizado interactivo de grafos dirigidos.

7.1 Vistas del Sistema

Vista (FXML)	Controlador	Funcionalidad
Principal.fxml	PrincipalController	Pantalla principal con navegación entre módulos
Menu.fxml	MenuController	Menú lateral de navegación entre vistas
Mapa.fxml	MapaController	Visualización interactiva del grafo y búsqueda de rutas
Paradas.fxml	ParadasController	Listado, búsqueda y eliminación de paradas
RegistroParada.fxml	RegistroParadaController	Formulario modal para agregar y modificar paradas
Rutas.fxml	RutasController	Listado, búsqueda y eliminación de rutas
RegistroRuta.fxml	RegistroRutaController	Formulario modal para agregar y modificar rutas

7.2 Vista del Mapa (MapaController)

El controlador del mapa es el más complejo del sistema. Sus responsabilidades incluyen:

- Construir el dígrafo visual de SmartGraph a partir del ServicioGrafo, insertando vértices y aristas con sus datos reales.
- Gestionar la selección de paradas mediante doble clic: el primer clic marca el origen (estilo azul claro), el segundo el destino (estilo azul oscuro).
- Ejecutar Dijkstra o Bellman-Ford según el algoritmo y criterio seleccionados
- Permitir la búsqueda de rutas alternativas acumulando los caminos previos y excluyéndolos en llamadas sucesivas.
- Actualizar las etiquetas de las aristas dinámicamente según el criterio activo (tiempo, costo, distancia o transbordos).

7.4 Formularios de Registro

Los formularios RegistroParadaController y RegistroRutaController son ventanas modales que se abren tanto para agregar como para modificar entidades. El formulario de ruta utiliza Spinners con validación de rangos para los cuatro criterios; en la versión final los rangos negativos están habilitados para soportar el caso de Bellman-Ford con pesos negativos. Ambos controladores validan los datos antes de delegar en las clases CRUD y muestran mensajes de error descriptivos mediante diálogos Alert de JavaFX.

8. Decisiones de Diseño

8.1 Singleton ServicioGrafo

En lugar de pasar el grafo como parámetro entre controladores, se adoptó el patrón Singleton a través de la clase ServicioGrafo. Esto garantiza que todos los controladores de JavaFX accedan siempre a la misma instancia del grafo, evitando inconsistencias entre vistas.

8.2 Copia del Grafo para Validación

Antes de aplicar modificaciones que puedan romper la conexidad del grafo (como cambiar el par origen-destino de una ruta), el sistema crea una copia superficial del grafo mediante copiar(), aplica la operación propuesta sobre la copia y verifica la conexidad. Solo si la copia pasa la validación se modifica el grafo real. Esto evita dejar el sistema en un estado inconsistente.

8.3 Listas de Adyacencia vs. Matriz de Adyacencia

Se optó por listas de adyacencia (DigraphEdgeList de SmartGraph) en lugar de matriz de adyacencia por tres razones: la red de transporte es típicamente dispersa (pocas rutas por parada), el acceso a los vecinos de un nodo es O(deg) —suficiente para los algoritmos de Dijkstra y BFS—, y la inserción y eliminación de aristas es eficiente.

8.4 Enum Criterio como clave de pesos

Los pesos de cada ruta se almacenan en un Map<Criterio, Object> dentro de la clase Ruta. Usar el enum Criterio como clave permite agregar nuevos criterios de optimización en el futuro sin modificar la firma de los métodos existentes; basta con añadir un nuevo valor al enum.

8.5 Arquitectura de Repositorios

La capa de persistencia fue diseñada con interfaces genéricas (RowMapper<T>, PreparedStatementMapper<T>) y un servicio genérico (StatementService<T>) para minimizar la duplicación de código. Cada entidad tiene su propio repositorio y su propio mapper, siguiendo el principio de responsabilidad única.

9. Pruebas y Validación

Esta sección documenta las pruebas funcionales y de validación realizadas sobre el sistema de rutas de transporte, aplicando el algoritmo Bellman-Ford sobre el grafo dirigido de 7 nodos con 13 aristas (3 de ellas con pesos negativos que representan descuentos, subsidios o bonificaciones en tiempo/costo).

9.1 Pruebas Funcionales

Se ejecutaron 13 casos de prueba cubriendo operaciones de inserción, validación, cálculo de rutas y persistencia. Los casos C1–C7 corresponden directamente a las pruebas del algoritmo Bellman-Ford con el grafo de referencia (nodos N1–N7).

Caso de Prueba	Resultado Esperado	Resultado Obtenido
Agregar parada con datos válidos	Parada aparece en la lista y en el mapa	Correcto
Agregar parada con nombre vacío	Mensaje de error; no se inserta	Correcto
Agregar ruta entre misma parada (A → A)	Mensaje de error; no se inserta	Correcto
Agregar ruta duplicada (mismo para A → B)	Mensaje de error; no se inserta	Correcto
Calcular ruta óptima por tiempo (C1: N1→N7)	Ruta resaltada: N1→N2→N4→N6→N7, costo 70 min	Correcto
Calcular ruta óptima por costo (C2: N1→N6)	Ruta resaltada: N1→N2→N4→N6, costo $28.00	Correcto
Ruta con arista negativa — subsidio (C3: N3→N7)	Bellman-Ford aplica N3→N2 (−10 min); ruta correcta	Correcto
Ruta mixta con retorno bonificado (C4: N4→N5)	N4→N1→N2→N5 con arista neg. N4→N1 (−20 min)	Correcto
Ruta alternativa distinta a la óptima (C5: N1→N4)	Segundo camino diferente al primero	Correcto
Ciclo negativo — detección (C7: N2→N1)	Sistema reporta ciclo negativo detectado; no calcula ruta inválida	Correcto
Modificar ruta que rompe la conexidad	Sistema rechaza y notifica parada problemática	Correcto
Eliminar parada con rutas asociadas	Parada y sus rutas eliminadas del grafo y la BD	Correcto
Persistencia entre sesiones	Datos cargados desde PostgreSQL al iniciar	Correcto

9.2 Pruebas de Validación de Grafos

Se validaron siete escenarios con grafos de diferente tamaño y topología. El grafo principal utilizado es un grafo dirigido de 7 nodos con conexidad total (todos los nodos poseen al menos una arista entrante y una saliente), garantizando la aplicabilidad del algoritmo.

Escenario	Topología / Caso	Comportamiento Esperado	Estado
Grafo lineal (cadena)	*N1→N2→N3→N4→N5→N6→N7*	Dijkstra & Bellman-Ford producen misma ruta	Correcto
Grafo circular (ciclo)	*N1→N2→N4→N6→N7→N1*	Se detecta el ciclo; no entra en bucle infinito	Correcto
Múltiples caminos (N1→N6)	*N1→N2→N4→N6 y N1→N3→N5→N6*	Ambos caminos válidos; se retorna el óptimo	Correcto
Grafo con pesos negativos (N3→N2, N4→N1, N6→N5)	*Bellman-Ford aplica relajación extra en aristas negativas*	Rutas C3, C4, C6 calculadas correctamente	Correcto
Detección de ciclo negativo	*Aristas negativas no forman ciclo de costo negativo total*	Sistema reporta ausencia de ciclo negativo; seguro continuar	Correcto
Parada aislada (sin conexiones)	*Sistema detecta nodo sin entrada o sin salida*	Operación de búsqueda rechazada con notificación	Correcto
Grafo 7 nodos — conexidad total	*Todos los nodos tienen ≥1 entrada y ≥1 salida*	Verificación de conexidad aprobada en todos los casos	Correcto

En todos los escenarios el algoritmo Bellman-Ford produjo resultados correctos. A diferencia de Dijkstra, Bellman-Ford maneja pesos negativos y detecta ciclos negativos (complejidad $O(|V|\cdot|E|) = O(91)$ para este grafo), lo que lo hace adecuado para rutas con tarifas subsidiadas o bonificaciones de tiempo. El sistema de verificación de conexidad funcionó como se esperaba, rechazando operaciones de búsqueda sobre paradas aisladas.

10. Conclusiones

El Sistema de Gestión de Rutas de Transporte Público desarrollado cumple con los requerimientos fundamentales establecidos en el documento del proyecto, logrando una solución funcional, visual e integrada con base de datos.

Entre los logros más destacados del proyecto se encuentran la implementación correcta de los algoritmos de Dijkstra, Bellman-Ford, Floyd-Warshall y Kruskal con soporte para múltiples criterios de optimización, la verificación de conexidad fuerte del dígrafo mediante BFS doble, la validación preventiva de operaciones que podrían romper la integridad del grafo, y la integración completa de una capa de persistencia con PostgreSQL que permite la continuidad entre sesiones.

La arquitectura adoptada, separación en capas de modelo, lógica, datos y vista, demostró ser adecuada para la escala del proyecto y facilita la incorporación de nuevas funcionalidades. En particular, el diseño genérico de los repositorios y mappers permite extender el esquema de datos con mínimas modificaciones al código existente.

Como trabajo futuro y áreas de mejora, se identifican: la incorporación de una capa de simulación en tiempo real con condiciones de tráfico dinámicas, la mejora en la visualización del Árbol de Expansión Mínima generado por Kruskal directamente sobre el mapa interactivo, y la optimización del rendimiento del sistema para redes de transporte de mayor escala.

En términos de aprendizaje, el proyecto permitió aplicar de manera práctica conceptos de estructuras de datos (grafos dirigidos, listas de adyacencia, colas de prioridad), algoritmos de búsqueda y optimización, patrones de diseño de software (Singleton, Repository, RowMapper), y desarrollo de interfaces gráficas con JavaFX.

12. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Oracle Corporation. (2023). JavaFX Documentation. https://openjfx.io/
- Bruno Mn Silva. (2022). SmartGraph: Interactive Graph Visualization for JavaFX. https://github.com/brunomnsilva/JavaFXSmartGraph
- PostgreSQL Global Development Group. (2023). PostgreSQL 15 Documentation. https://www.postgresql.org/docs/
- Apache Maven Project. (2023). Maven Documentation. https://maven.apache.org/
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.